

PROMPT PACK

Architecture Stress Test Prompt Pack

BUILD SYSTEMS, NOT JUST UI

Run these prompts before writing backend code. If you skip this, you are building UI, not systems.

00

SECTION 00

How to Use This Playbook

This playbook contains six architecture stress-test prompts designed to be executed **before** you write a single line of backend code. Each prompt targets a critical dimension of system design, from database schema integrity to cost projection and failure simulation. These are not optional checklists; they are the difference between building a prototype and building a production system that survives real-world load.

The prompts are formatted as ready-to-paste code blocks. Each one includes customizable placeholder fields marked in amber, which you should replace with your specific app details before running them through your preferred AI model. Do not accept the first answer from any AI model. Treat each response as a starting point for deeper investigation.

- 01 Replace the placeholder fields with your actual app idea and tech stack details
- 02 Paste one prompt at a time into your AI model of choice
- 03 Do NOT accept the first answer blindly; interrogate every recommendation
- 04 Ask targeted follow-up questions based on the AI's response
- 05 Refine the output until your system logic is crystal clear and battle-tested

The Golden Rule

If AI says "Looks good." — Ask "What would break at scale?"

Always interrogate the system. Always.

PROMPT 01

Schema Stress Test

Purpose: Design a strong, normalized database structure before connecting any frontend or API layer. This prompt forces you to think about data relationships at scale, identifying structural weaknesses that only emerge under production-level load with thousands of concurrent users.

COPY & PASTE THIS PROMPT

You are a senior backend architect.

Here is my app idea:

[PASTE YOUR FEATURE LIST HERE]

Step 1:

Design a normalized database schema including:

- Core entities
- Relationships
- Foreign keys
- Indexing suggestions

Step 2:

Simulate 10,000 daily active users.

Identify:

- Structural weaknesses
- Bottlenecks
- Redundant relationships
- Risk of write amplification

Step 3:

Suggest schema improvements for scale.

Be brutally honest.

Do not optimize for simplicity.

Optimize for long-term stability.

This prompt operates in three deliberate phases. The first phase establishes your baseline data model. The second phase applies stress by simulating realistic user volume, which exposes weaknesses like N+1 query patterns, missing composite indexes, and write amplification from denormalized data. The third

phase forces forward-looking architectural thinking, ensuring your schema can evolve without painful migrations.

PROMPT 02

Security Blind Spot Audit

Purpose: Prevent future "Oh sh*t" moments by systematically identifying security vulnerabilities in your architecture before they become exploits. This prompt covers the most common attack vectors that startups overlook during rapid development sprints.

COPY & PASTE THIS PROMPT

You are a backend security engineer.

Given this architecture:

[DESCRIBE YOUR STACK: Firebase / Supabase / Custom etc.]

Given this feature set:

[PASTE FEATURES]

List potential vulnerabilities including:

- Authentication bypass risks
- Improper role enforcement
- Rate abuse
- Injection attacks
- Data exposure risks
- Client-side trust issues

Then suggest mitigation strategies for each.

Security is rarely the first priority during early-stage development, which is exactly why it becomes the most expensive problem to fix later. This prompt creates a structured vulnerability audit that covers authentication, authorization, data exposure, and client-side trust boundaries. The mitigation strategies it generates give you a prioritized security roadmap, not just a list of scary possibilities. Run this early, run it often, and update it as your feature set evolves.

PROMPT 03

Vendor Lock-In & Migration Audit

Purpose: Think beyond MVP by evaluating the long-term migration challenges and vendor dependencies of your chosen technology stack. Most founders discover lock-in problems when they hit scale, by which point migration costs can kill the business.

COPY & PASTE THIS PROMPT

You are a systems architect evaluating long-term scalability.

If I build this app using:

[YOUR BACKEND CHOICE]

At 100k users:

- What migration challenges might arise?
- What parts of the system become hard to replace?
- What data portability issues may occur?
- What vendor limitations might hurt scale?

Suggest how to design today to reduce lock-in risk.

Vendor lock-in is the silent killer of scalable startups. Your BaaS provider may be perfect for 1,000 users, but at 100,000 users, proprietary query languages, limited export formats, and per-request pricing can strangle your growth. This prompt forces a forward-looking analysis of every dependency in your stack, from database providers to authentication services, helping you design abstraction layers and data portability strategies before they become emergency projects.

PROMPT 04

Cost Explosion Simulation

Purpose: Predict financial surprises before they happen by modeling cloud cost behavior at different growth stages. Many startups die not from lack of traction, but from unexpected infrastructure costs that outpace revenue growth.

COPY & PASTE THIS PROMPT

You are a cloud cost analyst.

Given this backend setup:

[STACK DETAILS]

Estimate cost behavior at:

- 100 DAU
- 1,000 DAU
- 10,000 DAU

Identify:

- Which metric drives cost (reads, writes, bandwidth, storage, function calls)
- What can spike unexpectedly
- Which architectural decisions increase cost sensitivity

Suggest optimizations for cost stability.

Cloud cost is rarely linear. A serverless function that costs pennies at 100 DAU can become a multi-thousand-dollar line item at 10,000 DAU due to cold starts, connection pooling limits, and data transfer charges. This prompt creates a three-stage cost model that identifies the specific metrics driving your bills at each growth phase, highlights potential spike scenarios like viral features or batch operations, and recommends architectural optimizations that keep costs predictable.

PROMPT 05

Failure Mode Simulation

Purpose: Understand what breaks first in your architecture under sudden traffic spikes. This is not about normal load; it is about the chaotic, unpredictable surges that happen when your product goes viral or experiences a flash crowd event.

COPY & PASTE THIS PROMPT

You are a distributed systems engineer.

Given this app architecture:

[DESCRIBE YOUR ARCHITECTURE]

Simulate sudden traffic spikes.

Answer:

- What fails first?
- What slows down first?
- What data becomes inconsistent?
- What monitoring should exist?

Then recommend architectural safeguards.

Normal load testing tells you if your system works under expected conditions. Failure mode simulation tells you what happens when everything goes wrong simultaneously: your authentication service times out, your database connection pool exhausts, and your CDN cache invalidates all at once. This prompt forces a systematic analysis of cascade failures, data consistency risks, and monitoring gaps, producing actionable safeguards like circuit breakers, rate limiters, and graceful degradation strategies.

PROMPT 06

Completion vs Stability Check

Purpose: Avoid the "App complete" illusion by evaluating whether your system design prioritizes feature velocity over long-term stability. This bonus prompt uncovers hidden technical debt that accumulates when teams optimize for shipping speed.

COPY & PASTE THIS PROMPT

Evaluate this system design.

Does it optimize for:

- Feature completion

or

- Long-term stability?

List:

- Hidden technical debt
- Unscalable shortcuts
- Poor separation of concerns

Be critical.

Assume this app will scale 100x.

The most dangerous phase of any product's lifecycle is the moment it feels "complete." That feeling often masks a mountain of technical debt: tightly coupled services, missing abstraction layers, hardcoded configuration values, and testing gaps that will make every future feature ten times harder to build. This prompt applies a 100x scaling lens to your current architecture, forcing you to confront the shortcuts you took to ship fast and evaluate whether they are sustainable.

FINAL RULE

Always Interrogate

If AI says "Looks good." — Ask "What would break at scale?"

Every comfortable answer is a trap. Every "looks good" hides a vulnerability you have not found yet. The difference between a system that survives and one that collapses is the depth of your questioning. Run every prompt. Ask every follow-up. Refine until the logic is unbreakable.